Week 3 作业：算法题、LP 松弛与 GPU

优化算法课程

说明：这周的核心想法是：

- 很多熟悉的算法题都能写成整数规划（ILP）；
- 把整数约束放松后，可以得到线性规划（LP）；
- LP 松弛后可能不再精确，但形式更规整，往往更适合 GPU 并行。

请尽量用自己的话解释：**原问题是什么、LP 松弛后变成了什么、误差从哪里来、为什么更适合 GPU**。

题 1 [基础]　一元线性回归中的梯度下降步长

背景： 考虑一元线性回归模型

$$\hat{y}_i = wx_i,$$

其中参数 $w \in \mathbb{R}$，训练数据为 (x_i, y_i)，$i = 1, \ldots, n$。平方损失函数定义为

$$f(w) = \frac{1}{2n} \sum_{i=1}^{n} (wx_i - y_i)^2.$$

我们用梯度下降更新参数：

$$w_{t+1} = w_t - \eta f'(w_t).$$

(a) 计算目标函数 $f(w)$ 的一阶导数 $f'(w)$ 和二阶导数 $f''(w)$。

(b) 设

$$A = \frac{1}{n} \sum_{i=1}^{n} x_i^2.$$

证明梯度下降更新可以写成

$$w_{t+1} - w^* = (1 - \eta A)(w_t - w^*),$$

其中 w^* 是最优解。

(c) 由上式说明：为了使梯度下降收敛，步长 η 至少应满足什么条件？请给出 η 的最大允许范围。

(d) 用自己的话解释：为什么当数据的 x_i 整体更大时，允许的最大步长会更小？这说明了步长与数据尺度之间有什么关系？

(e) （思考）若把所有输入同时放大为 $x_i' = cx_i$，其中 $c > 0$，那么允许的最大步长会如何变化？

题 2 [基础]　最大权二分匹配

背景： 有 n 个用户、m 个广告位。每个广告位最多分给一个用户，每个用户最多得到一个广告位。兼容度矩阵为 $C \in \mathbb{R}^{n\times m}$。平台希望最大化总兼容度。

(a) 用变量 x_{ij} 表示“用户 i 是否分配给广告位 j”，写出该问题的 ILP。

(b) 去掉整数约束，写出 LP 松弛。

(c) 用一句话说明：为什么这个问题的 LP 松弛比较“幸运”，通常不会带来误差？

题 3 [基础]　0-1 背包

背景： 有 n 个物品，第 i 个物品价值为 v_i，重量为 w_i，背包容量为 W。每个物品只能选或不选。

(a) 写出 0-1 背包的 ILP。

(b) 写出对应的 LP 松弛。

(c) 解释为什么 LP 松弛后可能出现“拿半个物品”的现象，因此 LP 最优值一般会比整数最优值更乐观。

题 4 [基础]　Vertex Cover

背景： 给定无向图 $G = (V, E)$。希望选出尽可能少的顶点，使得每条边至少有一个端点被选中。

(a) 写出 Vertex Cover 的 ILP。

(b) 写出对应的 LP 松弛。

(c) 若 LP 解中有些顶点取值是 1/2，这说明了什么？请用一句话解释“松弛误差”的含义。

题 5 [理解]　背包问题中的松弛误差

背景： 考虑如下 0-1 背包问题：共有 3 个物品，

$$(v_1, w_1) = (14, 6), \qquad (v_2, w_2) = (12, 5), \qquad (v_3, w_3) = (8, 4),$$

背包容量为 $W = 10$。设决策变量 $x_i \in \{0, 1\}$ 表示是否选择物品 i。

(a) 写出该问题的 ILP。

(b) 忽略整数约束，写出对应的 LP 松弛，并用“允许最后一个物品取分数”的方式，计算根节点的 LP 上界。

(c) 以变量 x_1 为第一个分支变量，分别考虑两个子问题：

  i. $x_1 = 1$；

  ii. $x_1 = 0$。

  对每个子问题，计算其 LP 松弛上界，并判断该分支是否可能产生更优整数解。

(d) 根据上面的结果，继续做一步分支定界，找出该背包问题的最优整数解。

(e) 用自己的话解释：分支定界法中的“分支”和“定界”分别是什么意思？为什么 LP 松弛可以用来帮助剪枝？

题 6 [理解]　Vertex Cover 的简单舍入

背景： 对 Vertex Cover 的 LP 松弛，考虑如下简单规则：若 $x_v^* \geq 1/2$，就把顶点 v 选入解中。

(a) 说明为什么这个规则一定得到一个合法的顶点覆盖。

(b) 证明该规则得到的解大小不超过 $2\sum_v x_v^*$。

(c) 用自己的话解释：为什么这说明 LP 松弛虽然不精确，但仍然能给出一个“还不错”的近似解？

题 7 [理解]　最短路为什么几乎没有松弛误差

背景： 在有向图中，最短路可以写成

$$\max d_t \qquad \text{s.t.} \quad d_v - d_u \leq w_{(u,v)}, \quad d_s = 0.$$

(a) 这里的变量 d_v 表示什么?

(b) 为什么这个模型本身已经是连续的，不像背包或 Vertex Cover 那样需要“选或不选”?

(c) 用一句话比较：为什么有些问题松弛后误差大，而最短路这种问题几乎没有这个问题?

题 8 [讨论]　从离散算法到规整迭代

背景： 很多 CPU 上的经典算法（如 Dijkstra、动态规划、分支搜索）都依赖复杂控制流；而 GPU 更擅长做大量结构相同的并行计算。

(a) 请比较下面两类计算的区别：

i. 用堆实现 Dijkstra；

ii. 对所有边同时做一次 Bellman-Ford 松弛。

哪一种更适合 GPU？为什么?

(b) 请比较下面两类计算的区别：

i. 用回溯法精确求解 0-1 背包：不断尝试“选 / 不选”每个物品；

ii. 将 0-1 背包放松成连续变量问题后，同时计算所有物品变量的更新方向。

哪一种更适合 GPU？为什么?

题 9 [讨论]　编辑距离与反对角线并行

背景： 编辑距离的经典动态规划为

$$\mathrm{dp}[i][j] = \min\Big(\mathrm{dp}[i-1][j]+1,\ \mathrm{dp}[i][j-1]+1,\ \mathrm{dp}[i-1][j-1]+\mathbf{1}[s_i \neq t_j]\Big).$$

(a) 解释为什么同一条反对角线上的状态可以同时更新。

(b) 说明这类“分层更新”的结构为什么适合 GPU。

(c) 用一句话说明：这道题虽然本质上是 DP，但为什么也能看成“更规整的并行松弛过程”?

题 10 [编程]　值迭代：CPU 与 GPU 对比

背景： 值迭代的核心更新为

$$V(s) \leftarrow \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right].$$

当对所有状态同时更新时，这一过程很适合向量化实现。

任务：在一个网格 MDP 上，实现 CPU 版和 GPU 版值迭代，并比较运行时间。

(a) 实现一个网格 MDP 生成函数，返回奖励矩阵 R 和转移张量 P。

(b) 用 NumPy 实现 CPU 版值迭代。

(c) 用 PyTorch 实现 GPU 版值迭代，并正确计时。

(d) 在不同网格规模下比较 CPU 与 GPU 的运行时间，并回答：

i. 状态数变大后，GPU 何时开始占优？

ii. 每轮迭代中，最耗时的部分通常是什么？

iii. 为什么值迭代比很多“带大量分支的搜索算法”更适合 GPU？

参考代码框架如下：

```python
import numpy as np
import torch
import time

def make_grid_mdp(grid_size, gamma=0.99, seed=42):
    """
    生成 grid_size x grid_size 的网格 MDP。
    返回：
        R: shape (S, A)
        P: shape (A, S, S)
    """
    pass

def value_iteration_cpu(R, P, gamma=0.99, tol=1e-6, max_iter=10000):
    S, A = R.shape
    V = np.zeros(S)
    start = time.time()
    for it in range(max_iter):
```

```python
        Q = R + gamma * np.einsum('aij,j->ia', P, V)
        V_new = Q.max(axis=1)
        if np.max(np.abs(V_new - V)) < tol:
            return V_new, it + 1, time.time() - start
        V = V_new
    return V, max_iter, time.time() - start

def value_iteration_gpu(R_np, P_np, gamma=0.99, tol=1e-6,
                        max_iter=10000, device='cuda'):
    R = torch.tensor(R_np, dtype=torch.float32, device=device)
    P = torch.tensor(P_np, dtype=torch.float32, device=device)
    V = torch.zeros(R_np.shape[0], dtype=torch.float32, device=
       device)

    torch.cuda.synchronize()
    start = time.time()

    for it in range(max_iter):
        Q = R + gamma * torch.einsum('aij,j->ia', P, V)
        V_new = Q.max(dim=1).values
        if torch.max(torch.abs(V_new - V)).item() < tol:
            torch.cuda.synchronize()
            return V_new.cpu().numpy(), it + 1, time.time() -
               start
        V = V_new

    torch.cuda.synchronize()
    return V.cpu().numpy(), max_iter, time.time() - start
```